PITCH CLOCK

A Presentation By
Zachary Benetzky
Brody Rebert

PROJECT OVERVIEW

- Pitch Clock
- Purpose
- Code
- Visuals
- Problems

PITCH CLOCK

- A rule implemented in the 2023 season of the MLB
- A timer that starts when pitcher receives the ball
- The timer is 15 seconds when nobody is on base
- The batter has until 8 seconds left

PURPOSE

- What player stats have changed from the implementation of the pitch clock?
- Has the pitch clock benefited pitchers or hitters more?

SETUP

```
import pandas as pd
import requests
import sqlalchemy
import notebook
from sqlalchemy import create_engine
from sqlalchemy import text
```

PLAYER IDS

```python
def get_all_player_ids(season):

    teams_url = "https://statsapi.mlb.com/api/v1/teams"
    params = {"sportId": 1, "season": season}

    response = requests.get(teams_url, params=params)
    response.raise_for_status()
    teams_data = response.json()

    all_players = {}

    for team in teams_data.get("teams", []):
        team_id = team["id"]
        team_name = team["name"]

        roster_url = f"https://statsapi.mlb.com/api/v1/teams/{team_id}/roster"
        roster_params = {"season": season}

        roster_response = requests.get(roster_url, params=roster_params)
        roster_response.raise_for_status()
        roster_data = roster_response.json()

        for player in roster_data.get("roster", []):
            player_id = player["person"]["id"]
            player_name = player["person"]["fullName"]

            if player_id not in all_players:
                all_players[player_id] = {
                    "name": player_name,
                    "team": team_name,
                    "team_id": team_id,
                    "position": player.get("position", {}).get("abbreviation", "N/A")
                }

    return all_players
```

```python
seasons = [2015, 2016, 2017, 2018, 2019, 2020, 2021, 2022, 2023, 2024, 2025]
all_players_dict = {}

for season in seasons:
    players = get_all_player_ids(season)
    print(f"Found {len(players)} active players")
    all_players_dict.update(players)

df_players = pd.DataFrame([
    {"player_id": player_id, **player_info}
    for player_id, player_info in all_players_dict.items()
])
```

```python
all_player_ids = df_players['player_id'].tolist()
```

PLAYER DATA

```python
def fetch_mlb_api(endpoint, **params):
    base_url = "https://statsapi.mlb.com/api/v1"
    url = f"{base_url}/{endpoint}"

    response = requests.get(url, params=params)
    response.raise_for_status()
    return response.json()

all_data = []

for season in seasons:
    for player in all_player_ids:
        endpoint = f"people/{player}/stats"
        params = {
            "stats": "season",
            "season": season,
            "gameType": "R"
        }
        player = fetch_mlb_api(endpoint, **params)
        player = extract_player_stats_to_dataframe(player)
        all_data.append(player)

mlb_data = pd.concat(all_data, ignore_index=True)
```

```python
def extract_player_stats_to_dataframe(api_response):
    if 'stats' in api_response and api_response['stats']:
        stats_data = api_response['stats'][0]

        if 'splits' in stats_data and stats_data['splits']:
            player_stats = stats_data['splits'][0]

            stat_dict = player_stats.get('stat', {})
            metadata = {
                'player_id': player_stats.get('player', {}).get('id'),
                'player_name': player_stats.get('player', {}).get('fullName'),
                'season': player_stats.get('season'),
                'team_id': player_stats.get('team', {}).get('id'),
                'team_name': player_stats.get('team', {}).get('name'),
                'league_id': player_stats.get('league', {}).get('id'),
                'league_name': player_stats.get('league', {}).get('name'),
                'game_type': player_stats.get('gameType'),
                'stat_type': stats_data.get('type', {}).get('displayName'),
                'stat_group': stats_data.get('group', {}).get('displayName')
            }

            combined_dict = {**metadata, **stat_dict}

            for key, value in combined_dict.items():
                if isinstance(value, str) and value.replace('.', '').replace('-', '').isdigit():
                    if '.' in value:
                        combined_dict[key] = float(value)
                    else:
                        combined_dict[key] = int(value)

            df = pd.DataFrame([combined_dict])

            return df

    return pd.DataFrame()
```

DATA INFORMATION

```
print(mlb_data.shape)
```

```
(15489, 77)
```

```
print(mlb_data.dtypes)
```

```
player_id              int64
player_name              str
season                 int64
team_id               object
team_name             object
                      ...
plateAppearances     float64
rbi                  float64
leftOnBase           float64
babip                 object
atBatsPerHomeRun      object
Length: 77, dtype: object
```

SPLITTING DATA

```
mlb_pitchers = mlb_data
```

```
mlb_pitchers = mlb_data.loc[mlb_data['stat_group'] == 'pitching']
print(mlb_pitchers.shape)
```

```
(8466, 77)
```

```
mlb_pitchers = mlb_pitchers.dropna(axis=1, how='all')
print(mlb_pitchers.shape)
```

```
(8466, 72)
```

SPLITTING DATA

```
mlb_hitters = mlb_data
```

```
mlb_hitters = mlb_data.loc[mlb_data['stat_group'] == 'hitting']
print(mlb_hitters.shape)
```

```
(7023, 77)
```

```
mlb_hitters = mlb_hitters.dropna(axis=1, how='all')
print(mlb_hitters.shape)
```

```
(7023, 44)
```

FIXING NULLS

```
mlb_hitters.loc[mlb_hitters['stolenBases'] == 0, 'stolenBasePercentage'] = None
```

```
mlb_hitters.loc[mlb_hitters['stolenBases'] == 0, 'caughtStealingPercentage'] = None
```

```
mlb_hitters.loc[mlb_hitters['homeRuns'] == 0, 'atBatsPerHomeRun'] = None
```

CREATING DATABASE

```
PG_URL = 'postgresql+psycopg2://postgres:postgres@localhost:5432/postgres'
engine = create_engine(PG_URL)
```

```
mlb_pitchers.to_sql('mlb_pitchers', engine, if_exists='replace', index=False)
```

```
mlb_hitters.to_sql('mlb_hitters', engine, if_exists='replace', index=False)
```

VISUALS

Link to Metabase

PROBLEMS

- Github
- Branches
- API
- Data
- Docker
- Metabase